CHRISTOPHER NEWPORT UNIVERSITY · SCHOOL OF ENGINEERING & COMPUTING

A basic introduction to

Retrieval-Augmented Generation

Enhancing LLMs with external knowledge

Keith Perkins · School of Engineering & Computing

THE PROBLEM

The fundamental limitations of LLMs



While LLMs have revolutionized NLP, a plain prompt-in, response-out model faces several critical constraints:

Knowledge cutoff

Training data is frozen at a specific date, so the model doesn't know about recent events, updates, or changes.

Hallucination risk

Models can generate plausible-sounding but factually incorrect information, with confidence.

Context window costs

Processing long prompts is computationally expensive and slows responses, so you can't just paste every document into the prompt.

Domain knowledge gaps

Models have no access to private company data, proprietary documents, or specialized knowledge missing from their training data.

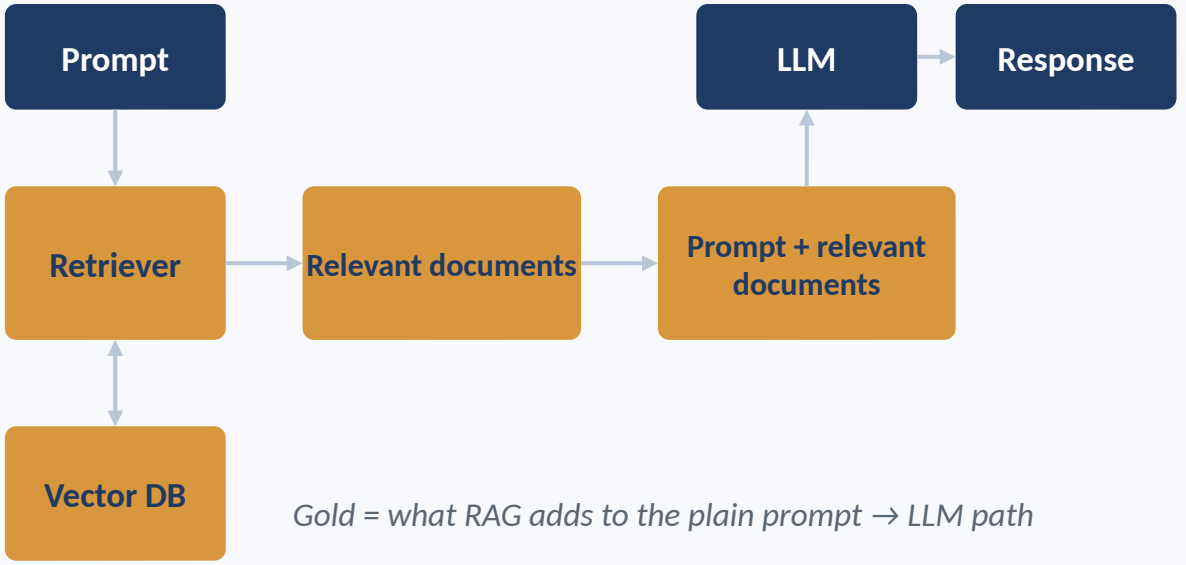
THE IDEA

What is Retrieval-Augmented Generation?

RAG gives an LLM information it doesn't have from training. It retrieves relevant information from a knowledge store and adds it to the prompt during generation.

CRITICAL DISTINCTION

RAG is not fine-tuning. No weights change. Relevant external information is added to the model's prompt **at inference time**, so the model can ground its answer in retrieved facts.



It separates the model's parametric knowledge (stored in its weights) from a knowledge base you choose: update the documents without retraining, and see which sources were used.

THE PARTS

Core components of a RAG system

01 Knowledge base / document store

A collection of documents, passages, or structured data that serves as external memory: company wikis, research papers, product documentation, or any domain-specific corpus.

02 Embedding model + vector index

A neural encoder that turns text into dense vectors, stored in an index built for fast similarity search.

03 Retrieval engine (top-k search)

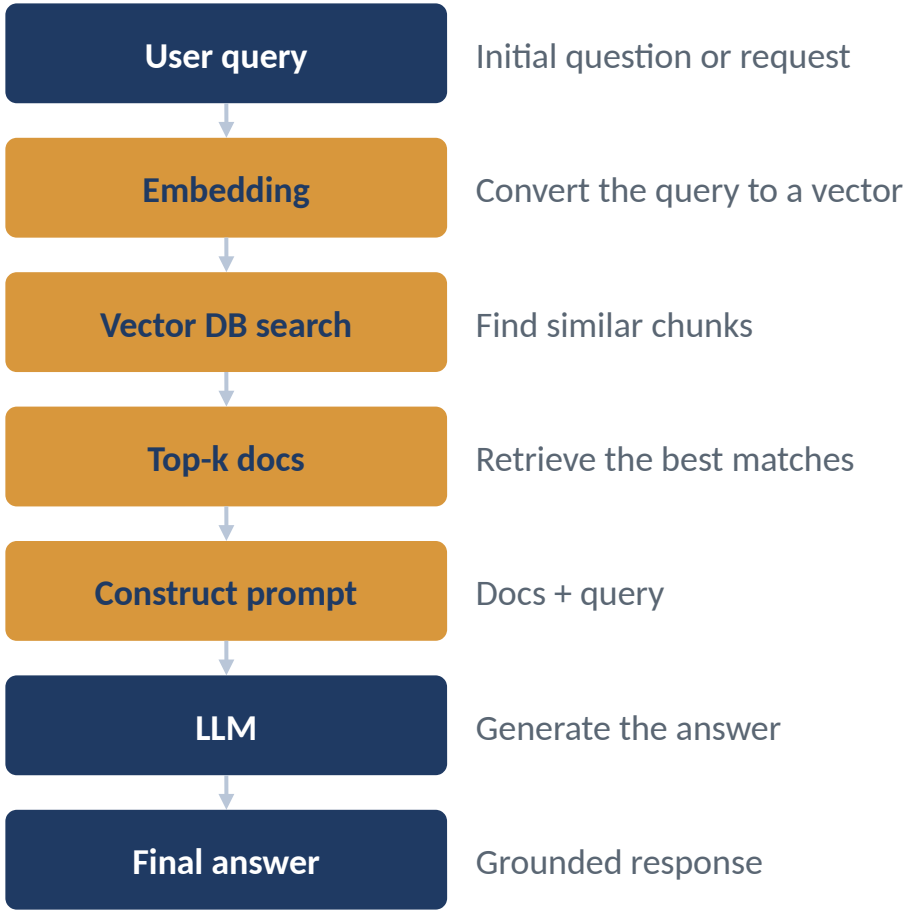
Finds the k documents or passages whose vectors are most similar to the query's vector.

04 Prompt construction + LLM generation

Assembles the retrieved passages and the user's question into a structured prompt, then the LLM generates the final answer.

STEP BY STEP

RAG architecture flow



Each step in this pipeline is critical.

- The query must be embedded with the same model as the documents
- The search must return truly relevant passages
- The prompt must guide the LLM to use the retrieved information appropriately

STEP BY STEP

Two phases: build the index, then answer questions

1 · Indexing

Offline: once, and again whenever the documents change

Load
Read the documents

→

Chunk
Split into small, overlapping pieces

→

Embed
One vector per chunk

→

Store
Vectors + chunk text in a vector DB

2 · Querying

Online: every time a user asks a question

Embed query
Same embedding model as indexing

→

Search
Find the top-k most similar chunks

→

Build prompt
Chunks + question

→

Generate
LLM writes a grounded answer

The previous slide's flow is phase 2. Phase 1 has to happen first.

KEY CONCEPT

Understanding embeddings

1 Dense vector mapping

Text becomes a vector of typically 384–3,072 numbers that captures its meaning.

2 Semantic similarity

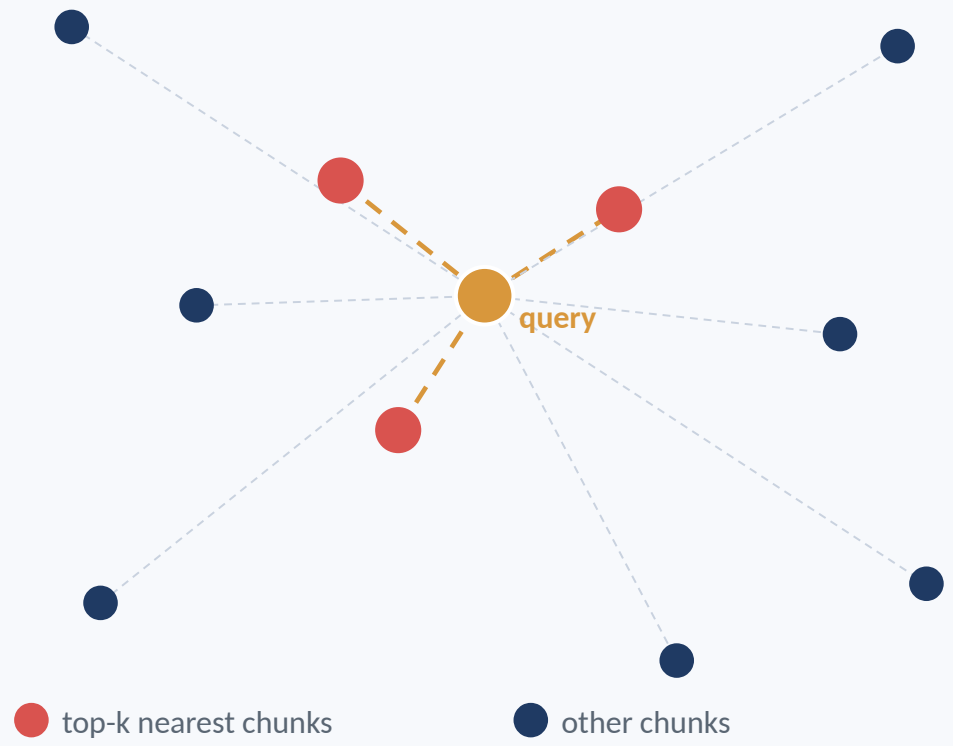
Texts with similar meanings land close together, even with no words in common.

3 Fast similarity search

Cosine similarity or dot product between vectors; specialized indexes search millions of vectors in milliseconds.

Embedding models are transformer encoders: contextual token vectors, combined into one vector per passage.

RETRIEVAL IN VECTOR SPACE (2-D SKETCH)



KEY CONCEPT · BY HAND

Cosine similarity, with numbers

THE FORMULA

$$\cos(a, b) = (a \cdot b) \div (|a| \times |b|)$$

a · b multiply matching entries, then add

|a| length: square root of the sum of squares

Result: 1 = same direction (same meaning), 0 = unrelated

If vectors are normalized to length 1, cosine is just the dot product.

TOY 3-D EMBEDDINGS: DIMENSIONS = [PETS, SPORTS, MONEY]

query	*"my cat"*	`[0.9, 0.1, 0.0]`
chunk A	*"dogs and cats"*	`[0.8, 0.2, 0.1]`
chunk B	*"stock prices"*	`[0.0, 0.1, 0.95]`

query vs A — **Very similar: retrieve it**

```
dot: 0.9·0.8 + 0.1·0.2 + 0·0.1 = 0.74
cos: 0.74 ÷ (0.906 × 0.831) = 0.98
```

query vs B — **Unrelated: skip it**

```
dot: 0.9·0 + 0.1·0.1 + 0·0.95 = 0.01
cos: 0.01 ÷ (0.906 × 0.955) = 0.01
```

INFRASTRUCTURE

Vector database technology

Modern RAG systems rely on specialized tools optimized for high-dimensional similarity search. We'll study how they search so fast next week.

FAISS

LIBRARY

Meta's (Facebook's) library for efficient similarity search, with many index types and GPU acceleration for massive scale.

Chroma

OPEN SOURCE

Developer-friendly embedding database designed for easy use from Python.

Pinecone

MANAGED CLOUD

Hosted vector database with automatic scaling and low-latency search.

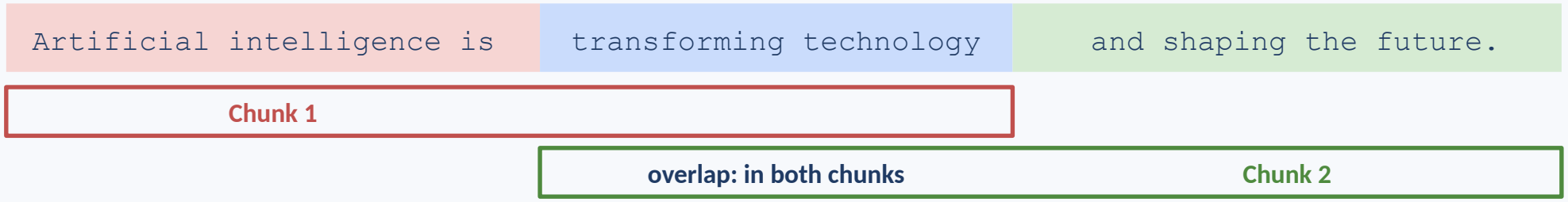
Milvus

OPEN SOURCE

Vector database built for production, with distributed architecture support.

Key trade-offs: query speed vs. index size and memory. Approximate nearest neighbor (ANN) search gives up a little accuracy for a dramatic speedup. We'll study this algorithm next week.

Document chunking strategy



WHY CHUNKING MATTERS

A long document covers many topics. One vector for the whole thing is too general to match a specific question, so we split documents into smaller, focused chunks. Fixed-size chunking is the simplest of many strategies.

Documents too long

They exceed the embedding model's best length and blur several topics together.

Split into chunks

Smaller segments (256–512 tokens is typical), each ideally about one idea.

Add overlap

10–20% overlap between neighbors so nothing important is cut in half at a boundary.

Advanced strategies

Semantic chunking splits at natural topic boundaries; it can beat fixed-size splits, but costs more.

Constructing effective prompts

EXAMPLE TEMPLATE

```
You are a helpful assistant. Answer the user's
question **using only** the provided context.
If the answer isn't in the context, say you don't know.

Question: {user_question}

Context:
{context}

Instructions:
- Ground your answer in the context.
- If the context is insufficient, say "I don't know
  based on the provided context."
- Cite the chunk indices you used
  (e.g., [CHUNK 0, CHUNK 2]).
```

THE ASSEMBLY PROCESS

Retrieved passages go into the prompt in clear sections. The instructions steer the model to answer from that context and cite it.

CRITICAL CONSIDERATIONS

- Stay within the context window, and remember longer prompts cost more and run slower
- Put the most relevant passages first or last: models use the middle of long prompts least well
- Say "answer based only on the provided context"
- Ask for source citations

WORKED EXAMPLE

One question, start to finish

Question: When is the final project due?

STEPS 1–2 · EMBED THE QUESTION, SCORE EVERY CHUNK

[CHUNK 2] The final project is due December 12 and is worth 30% of the grade. **0.71**

[CHUNK 0] Late homework loses 10% per day, up to 3 days. **0.28**

[CHUNK 1] Office hours are Tuesday and Thursday, 2–4 p.m. **0.09**

Gold border = top-2 (k = 2)

STEP 3 · THE FILLED-IN PROMPT

```
system:
Answer using only the context. Cite chunks.
If the answer isn't there, say you don't know.
user:
Context:
[CHUNK 2] The final project is due
  December 12 and is worth 30%...
[CHUNK 0] Late homework loses 10%...
Question: When is the final project due?
```

STEP 4 · ANSWER

"The final project is due December 12 [CHUNK 2]."

TRY IT

RAG in about 20 lines of Python

```
from sentence_transformers import SentenceTransformer
from transformers import pipeline

chunks = ["Late homework loses 10% per day, up to 3 days.",
          "Office hours are Tuesday and Thursday, 2-4 p.m.",
          "The final project is due December 12 ..."]

# Phase 1: indexing
embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
chunk_vecs = embedder.encode(chunks, normalize_embeddings=True)

# Phase 2: querying
question = "When is the final project due?"
scores = chunk_vecs @ embedder.encode(question, normalize_embeddings=True)
top = scores.argsort()[::-1][:2]                # top-k, k = 2
context = "\n".join(f"[CHUNK {i}] {chunks[i]}" for i in top)

llm = pipeline("text-generation", model="Qwen/Qwen2.5-0.5B-Instruct")
system = "Answer using only the context. Cite chunks."
user = f"Context:\n{context}\n\nQuestion: {question}"
messages = [{"role": "system", "content": system},
            {"role": "user", "content": user}]
print(llm(messages, max_new_tokens=80)[0]["generated_text"][-1]["content"])
```

Embedding model

all-MiniLM-L6-v2: small, fast, 384-number vectors. It reads only the first 256 tokens, so chunks must be short.

Normalize, then dot

Normalized vectors make cosine similarity a plain dot product: one matrix multiply scores every chunk.

Same LLM as before

The local Qwen pipeline and messages format from the prompting notebook.

APPLICATIONS

Real-world use cases for RAG

Q&A over private company docs

Employees query internal documentation, policies, procedures, and knowledge bases in plain language: less time searching, more access to information.

Chatbots with up-to-date knowledge

Customer service bots that use current product information, troubleshooting guides, and FAQs. Update the knowledge base without retraining the model.

Domain-specific assistants

Legal, medical, financial, or technical assistants grounded in authoritative sources like case law, medical literature, or regulations.

Long-form document summarization

Summarize long reports or collections by retrieving the relevant sections and combining them into a summary faithful to the original.

CHOOSING AN APPROACH

RAG vs fine-tuning

Rule of thumb: RAG for knowledge, fine-tuning for behavior (format, style, task skills). Many systems use both.

RAG

STRENGTHS

- ✓ Fast to update: just add new documents
- ✓ Transparent sources, easier to explain
- ✓ Access to external, current knowledge
- ✓ Lower upfront cost: no training

WEAKNESSES

- ✗ Retrieval errors carry into the answer
- ✗ Extra latency from the search step
- ✗ Only as good as the retrieval

Fine-tuning

STRENGTHS

- ✓ Teaches behavior, format, and style
- ✓ Better task-specific performance
- ✓ No retrieval latency
- ✓ Consistent behavior patterns

WEAKNESSES

- ✗ Poor at adding new facts reliably
- ✗ Static knowledge: retrain to update
- ✗ Expensive compute requirements
- ✗ Risk of catastrophic forgetting

KNOW THE LIMITS

Fundamental limitations of RAG

RAG introduces new ways to fail that you need to understand before deploying it.

1. **Retrieval noise**
   Search can return irrelevant or loosely related passages, adding misleading context and hurting the answer. Ambiguous queries make this worse.

2. **Persistent hallucinations**
   Even with retrieved documents, the model can still hallucinate if the context is incomplete, contradictory, or unclear, blending retrieved facts with made-up ones.

3. **Index maintenance costs**
   Keeping the vector index in sync with source documents means re-embedding and re-indexing. For large, frequently updated collections that's a real operational burden.

4. **Context window limits**
   Retrieved passages use up prompt space. More passages means higher cost and latency, and models use the middle of a long prompt least well.

SUMMARY

Key takeaways

RAG = retrieve + generate

Augment an LLM with external knowledge: retrieve relevant information, then use it during generation.

Embeddings + vector search

Dense embeddings and similarity search make retrieval fast at scale and match meaning, not just keywords.

Context through prompting

Retrieved passages go into the prompt as context, steering the LLM toward grounded answers.

Trade-offs matter

Compared with fine-tuning: faster updates and current knowledge vs. built-in, task-specific performance.

Next: vector search algorithms (ANN, HNSW), advanced chunking, the Weaviate and Chroma vector databases, and measuring retrieval quality with precision, recall, and MRR.

Prepare: install Chroma and pick 10–50 sample documents to try different chunking approaches.